Building a Searchable Paper Library

Local RAG: PDFs $\rightarrow$ Summaries $\rightarrow$ Embeddings $\rightarrow$ Search

Jake Anderson

UCLA

March 2026

The Idea

Problem: You have a folder of PDFs. You want to ask:

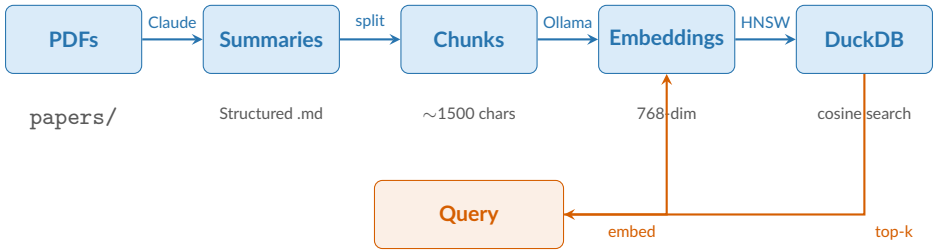
- "What papers use judge IV designs to study sentencing?"
- "Which papers find that incarceration increases recidivism?"

Solution: A local RAG pipeline:

1. Generate **structured summaries** of each paper (via Claude)
2. **Chunk** the summaries into passages
3. **Embed** each chunk locally (Ollama + nomic-embed-text)
4. Store embeddings in **DuckDB** with an HNSW index
5. Search by cosine similarity

No cloud services needed for search. Everything runs on your laptop.

Architecture



Ingestion (one-time)

`rag_ingest.py` — chunk, embed, store

Search (instant)

`search.py` — embed query, find neighbors

Prerequisites

Software

1. **Python 3.10+**
2. **Ollama** — local embeddings
   - `brew install ollama`
   - `ollama pull nomic-embed-text`
3. **Claude Code** — summaries
   - Needs an Anthropic account

Python packages

- `duckdb` — vector store
- `ollama` — Python client
- `python-frontmatter` — YAML parsing

```
pip install duckdb ollama
    python-frontmatter
```

Embedding model is **274 MB**, runs locally. No API costs for search.

Step 1: Generate Summaries

Open Claude Code in your project directory:

- Ask it to read each PDF and write a summary
- Use subagents to parallelize (6–8 papers per agent)
- Each summary has YAML frontmatter + standardized sections

YAML Header

- Title, Authors, Year
- Journal, Tags, Location
- DOI, Source file

Body Sections

- Research Question
- Econometric Strategy
- Identification
- Key Findings
- Limitations

See `setup/setup_prompt.md` for the exact prompt to give your LLM.

Step 2: Build the Vector Store

`rag_ingest.py` does three things:

1. **Chunk** — splits each summary at `##` headings
   - ~1500 chars per chunk, paragraph-boundary aware
   - Prepends author/title/year context to each chunk
2. **Embed** — sends chunks to Ollama in batches of 50
   - `nomic-embed-text`: 768-dim vectors
   - Uses `search_document:` prefix per model spec
3. **Store** — writes to DuckDB with HNSW index
   - Uses the `vss` extension for vector similarity

Commands

```
ollama serve &    (if not already running)

python scripts/rag_ingest.py
```

Step 3: The `papers` Command

Add a shell function to `~/.zshrc`:

```
papers() {
  /path/to/project/.venv/bin/python \
  /path/to/project/scripts/search.py "$@"
}
```

Then search from any terminal:

Basic

```
papers "judge leniency IV recidivism"
```

With filters

```
papers "police cameras" --author Chalfin

papers "pretrial detention" --tag bail

papers "drug sentencing" --year 2024 -n 5 -v
```

Results grouped by paper: authors, title, similarity, then chunks.

Adding New Papers

1. Copy PDFs to `papers/` with naming: `YEAR-title-slug.pdf`
2. Generate summaries (Claude Code or manually)
3. Re-run: `python scripts/rag_ingest.py`
   - Rebuilds the full vector store ($\sim$30 sec for 400 papers)

Current scale

248 PDFs, 395 summaries, $\sim$4000 chunks.
DuckDB file: 1.4 GB. Search: $<$1 second.

Quick start

The `setup/` folder contains everything you need:
`rag_ingest.py`, `search.py`, and a full setup prompt
you can paste into Claude Code to replicate this on your machine.